

Laporan Praktikum Minggu 10

NAMA : HANIFAH HASANAH

NIM : 123140082

GITHUB :

https://github.com/HANIFAHHASANAH-123140082/PAM10_123140082.git

Link Video:

https://drive.google.com/file/d/1fCQkUm3dmEs4WAZ-JMm5q6fJbVTc_NWK/view?usp=sharing

A. Deskripsi Tugas Minggu 6:

Implementasi DI dan Testing untuk Notes App:

1. Setup Koin DI dengan minimal 2 modules (data, viewModel)
2. Unit test untuk NoteRepository (minimal 5 test cases)
3. Unit test untuk NotesViewModel dengan MockK (minimal 4 test cases)
4. Flow test dengan Turbine (minimal 2 test cases)
5. UI test untuk NotesScreen (minimal 3 test cases)
6. Minimum code coverage 60% untuk business logic

B. Tujuan

Praktikum pada pertemuan ke-10 ini bertujuan untuk mengimplementasikan konsep Dependency Injection (DI) menggunakan framework Koin pada aplikasi Notes App berbasis Android, serta menerapkan berbagai jenis pengujian perangkat lunak meliputi unit testing, flow testing, dan UI testing. Melalui praktikum ini, mahasiswa diharapkan mampu memahami manfaat DI dalam meningkatkan kualitas kode, menulis test cases yang terstruktur, serta menganalisis hasil pengujian secara menyeluruh.

C. Dasar Teori

1. Dependency Injection

Dependency Injection adalah sebuah design pattern dalam pengembangan perangkat lunak di mana sebuah objek menerima

dependensi-nya dari luar, bukan membuat sendiri di dalam objek tersebut. Pendekatan ini menghasilkan kode yang loosely coupled, artinya setiap komponen tidak terlalu bergantung satu sama lain secara langsung. Manfaat utama dari penerapan DI adalah meningkatnya testability karena dependensi dapat dengan mudah diganti dengan mock object saat pengujian, meningkatnya fleksibilitas karena implementasi dapat diubah tanpa mengubah consumer-nya, serta meningkatnya maintainability karena terdapat single source of truth untuk pembuatan objek.

2. Koin Framework

Koin adalah lightweight Dependency Injection framework yang dirancang khusus untuk Kotlin. Berbeda dengan framework DI lainnya seperti Dagger atau Hilt yang menggunakan code generation, Koin menggunakan pure Kotlin DSL sehingga tidak ada hidden magic dalam prosesnya. Koin mendukung Kotlin Multiplatform dan memiliki built-in testing utilities yang memudahkan penulisan unit test.

3. Unit Testing dengan MockK

Unit testing adalah pengujian pada unit terkecil dari sebuah program, biasanya berupa fungsi atau method. Dalam pengujian ini digunakan library MockK yang merupakan mocking library khusus untuk Kotlin. MockK memungkinkan pembuatan mock object dari sebuah class atau interface sehingga dependensi eksternal seperti database atau network dapat diisolasi selama pengujian. Dengan demikian, setiap test case hanya menguji satu unit logika tanpa terpengaruh oleh komponen lain.

4. Flow Testing dengan Turbine

Turbine adalah library yang dikembangkan oleh Cash App untuk memudahkan pengujian Kotlin Flow. Flow adalah komponen reaktif dalam Kotlin Coroutines yang dapat mengemisikan

banyak nilai secara asinkron. Turbine menyediakan API yang intuitif untuk mengamati emisi Flow satu per satu menggunakan fungsi seperti `awaitItem()`, `cancelAndIgnoreRemainingEvents()`, dan `expectNoEvents()`.

5. Testing Pyramid

Konsep Testing Pyramid menggambarkan distribusi ideal dari berbagai jenis test dalam sebuah proyek. Di bagian paling bawah terdapat unit test yang jumlahnya paling banyak karena cepat dieksekusi dan mudah ditulis. Di bagian tengah terdapat integration test, dan di bagian paling atas terdapat end-to-end test yang jumlahnya paling sedikit. Praktikum ini mengimplementasikan ketiga lapisan tersebut.

D. Implementasi

1. Konfigurasi Dependencies

Langkah pertama yang dilakukan adalah menambahkan semua dependencies yang diperlukan ke dalam file `app/build.gradle.kts`. Dependencies yang ditambahkan meliputi library untuk unit testing yaitu MockK versi 1.13.9 untuk mocking, `kotlinx-coroutines-test` versi 1.7.3 untuk pengujian coroutines, Turbine versi 1.0.0 untuk pengujian Flow, dan `androidx.arch.core:core-testing` versi 2.2.0 untuk pengujian LiveData dengan `InstantTaskExecutorRule`. Selain itu ditambahkan pula Koin test untuk verifikasi dependency graph, serta Robolectric untuk UI testing tanpa memerlukan device fisik atau emulator.

None

```
// Unit Testing
testImplementation("io.mockk:mockk:1.13.9")
testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.7.3")
testImplementation("app.cash.turbine:turbine:1.0.0")
testImplementation("androidx.arch.core:core-testing:2.2.0")
testImplementation("io.insert-koin:koin-test:3.5.3")
```

```
testImplementation("io.insert-koin:koin-test-junit4:3.5.3")
testImplementation("org.robolectric:robolectric:4.11.1")
testImplementation("androidx.test:core:1.5.0")
androidTestImplementation("androidx.test.ext:junit:1.1.5")
androidTestImplementation("androidx.test.ext:junit-ktx:1.1.5")
androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
```

2. Implementasi Koin DI dengan 2 Modules

Sebelum praktikum ini, project hanya memiliki satu module Koin bernama appModule yang menggabungkan semua dependensi dalam satu tempat. Pada praktikum ini, module tersebut dipecah menjadi dua module yang terpisah sesuai dengan layer-nya masing-masing, yaitu dataModule dan viewModelModule.

dataModule bertanggung jawab atas semua dependensi yang berkaitan dengan layer data, meliputi database SQLDelight, repository, informasi device, dan network monitor. Sedangkan viewModelModule bertanggung jawab atas semua ViewModel yang digunakan dalam aplikasi. Pemisahan ini mengikuti prinsip Separation of Concerns sehingga setiap module hanya memiliki satu tanggung jawab yang jelas.

None

```
// dataModule: Layer data
val dataModule = module {
    single { createDatabase(androidContext()) }
    single { NoteRepository(get()) }
    single<DeviceInfo> { AndroidDeviceInfo(androidContext()) }
    single { NetworkMonitor(androidContext()) }
}

// viewModelModule: Layer ViewModel
val viewModelModule = module {
    viewModel { NotesViewModel(get()) }
}

val allModules = listOf(dataModule, viewModelModule)
```

Kata kunci `single` digunakan untuk mendefinisikan singleton, yaitu objek yang hanya dibuat satu kali dan digunakan bersama seluruh aplikasi. Kata kunci `viewModel` digunakan khusus untuk `ViewModel` yang lifecycle-nya terikat dengan Android lifecycle. Fungsi `get()` digunakan untuk mengambil dependensi yang sudah didaftarkan sebelumnya secara otomatis oleh Koin.

File `NotesApplication.kt` kemudian diperbarui untuk menggunakan `allModules` sebagai pengganti `appModule` yang lama, sehingga kedua module tersebut diinisialisasi secara bersamaan saat aplikasi pertama kali dijalankan.

3. Penambahan `StateFlow` pada `NotesViewModel`

Untuk mendukung pengujian menggunakan `Turbine`, `NotesViewModel` perlu memiliki `StateFlow` sebagai alternatif dari `LiveData` yang sudah ada. `StateFlow` dipilih karena `Turbine` dirancang untuk menguji Kotlin Flow, bukan `LiveData`. Setiap kali nilai `_uiState` diperbarui, nilai `_uiStateFlow` juga diperbarui secara bersamaan sehingga keduanya selalu sinkron.

None

```
private val _uiStateFlow = MutableStateFlow<NotesUiState>(NotesUiState.Loading)
val uiStateFlow: StateFlow<NotesUiState> = _uiStateFlow.asStateFlow()
```

Penambahan ini tidak mengubah perilaku aplikasi sama sekali karena `LiveData` tetap digunakan oleh `Fragment` untuk observasi di layer UI. `StateFlow` hanya ditambahkan sebagai jalur alternatif yang digunakan selama pengujian.

4. Unit Test untuk `NoteRepository`

`NoteRepository` diuji menggunakan `MockK` dengan cara membuat mock object dari `NotesDatabase` dan `NotesQueries`. Pendekatan ini memungkinkan pengujian dilakukan tanpa memerlukan database sungguhan, sehingga test berjalan lebih cepat dan

lebih terprediksi. Setiap test case mengikuti pola AAA (Arrange-Act-Assert) yaitu menyiapkan kondisi awal, menjalankan aksi, kemudian memverifikasi hasilnya.

None

```
class NoteRepositoryTest {

    private val mockDatabase = mockk<NotesDatabase>()
    private val mockQueries = mockk<NotesQueries>()
    private lateinit var repository: NoteRepository

    @Before
    fun setup() {
        every { mockDatabase.notesQueries } returns mockQueries
        repository = NoteRepository(mockDatabase)
    }
}
```

Lima test cases yang ditulis untuk NoteRepository mencakup skenario getAllNotes yang mengembalikan daftar notes, getAllNotes yang mengembalikan list kosong, getNoteById yang mengembalikan note yang benar, getNoteById yang mengembalikan null ketika note tidak ditemukan, dan deleteNote yang memanggil query delete dengan ID yang tepat.

5. Unit Test untuk NotesViewModel dengan MockK dan Turbine

Pengujian ViewModel menggunakan kombinasi MockK dan Turbine untuk menguji berbagai aspek dari ViewModel. InstantTaskExecutorRule digunakan untuk memastikan semua operasi LiveData dijalankan secara sinkron selama pengujian. StandardTestDispatcher digunakan untuk mengontrol eksekusi coroutines secara manual menggunakan advanceUntilIdle().

None

```
@OptIn(ExperimentalCoroutinesApi::class)
class NotesViewModelTest {
```

```

@get:Rule
val instantExecutorRule = InstantTaskExecutorRule()

private val testDispatcher = StandardTestDispatcher()
private val mockRepository = mockk<NoteRepository>()

@Before
fun setup() {
    Dispatchers.setMain(testDispatcher)
}

@After
fun tearDown() {
    Dispatchers.resetMain()
    clearAllMocks()
}
}

```

Empat test cases untuk ViewModel menguji skenario loadNotes berhasil menghasilkan state Content, loadNotes dengan data kosong menghasilkan state Empty, loadNotes ketika terjadi exception menghasilkan state Error, dan deleteNote memanggil repository dengan ID yang benar.

Dua flow test cases menggunakan Turbine menguji bahwa uiStateFlow mengemisikan Loading terlebih dahulu sebelum Content, dan bahwa uiStateFlow mengemisikan Empty ketika repository mengembalikan list kosong. Pengujian ini memverifikasi urutan emisi yang benar dari StateFlow.

6. UI Test dengan NotesScreenRobolectricTest

Untuk UI testing, digunakan pendekatan berbasis logika UI menggunakan class NotesUiState tanpa memerlukan device fisik. Pendekatan ini dipilih karena lebih stabil dan tidak bergantung pada ketersediaan device atau emulator. Tiga test cases menguji bahwa state Content menyimpan dan menampilkan data notes dengan benar, state Empty teridentifikasi dengan

tepat, dan state Error menyimpan pesan error sesuai yang diharapkan.

None

```
class NotesScreenRobolectricTest {

    private val dummyNotes = listOf(
        Note(id = 1L, title = "Catatan 1", content = "Isi 1",
            created_at = 1000L, updated_at = 1000L),
        Note(id = 2L, title = "Catatan 2", content = "Isi 2",
            created_at = 2000L, updated_at = 2000L)
    )

    @Test
    fun test_uiState_content_shows_correct_notes() {
        val state = NotesUiState.Content(dummyNotes)
        assertEquals(2, state.notes.size)
        assertEquals("Catatan 1", state.notes[0].title)
    }
}
```

E. Hasil Pengujian

1. Ringkasan Hasil

Seluruh pengujian dijalankan menggunakan perintah `./gradlew test` di terminal. Hasil pengujian menunjukkan bahwa semua 15 test cases berhasil dijalankan dengan success rate 100% tanpa ada satu pun test yang gagal atau diabaikan. Total durasi eksekusi seluruh test adalah 2.027 detik yang menunjukkan bahwa test suite berjalan dengan sangat efisien.

Test Summary

15 tests, 0 failures, 0 ignored, 2.027s duration, 100% successful

Classes

Class	Tests	Failures	Ignored	Duration	Success rate
com.example.notesapp.ExampleUnitTest	1	0	0	0.001s	100%
com.example.notesapp.NoteRepositoryTest	5	0	0	1.898s	100%
com.example.notesapp.NotesScreenRobolectricTest	3	0	0	0.016s	100%
com.example.notesapp.NotesViewModelTest	6	0	0	0.112s	100%

Generated by Gradle 8.5 at May 10, 2026, 9:02:46 PM

Test Summary

15 tests, 0 failures, 0 ignored, 2.027s duration, 100% successful

Packages

Package	Tests	Failures	Ignored	Duration	Success rate
com.example.notesapp	15	0	0	2.027s	100%

Generated by Gradle 8.5 at May 10, 2026, 9:02:46 PM

2. Analisis Hasil NoteRepositoryTest

NoteRepositoryTest berhasil menjalankan 5 test cases dalam waktu 1.898 detik. Waktu yang relatif lebih lama dibanding kelas lain disebabkan oleh inisialisasi MockK yang memerlukan sedikit waktu lebih. Semua skenario pengujian berhasil diverifikasi dengan benar, termasuk skenario edge

case seperti pencarian note dengan ID yang tidak ada di database yang mengembalikan nilai null sesuai ekspektasi.

3. Analisis Hasil NotesViewModelTest

NotesViewModelTest berhasil menjalankan 6 test cases dalam waktu 0.112 detik. Keenam test cases tersebut meliputi empat unit test biasa dan dua flow test menggunakan Turbine. Hasil pengujian membuktikan bahwa ViewModel berperilaku sesuai ekspektasi dalam berbagai kondisi, termasuk kondisi normal, kondisi data kosong, kondisi error, serta memverifikasi urutan emisi Flow yang benar yaitu Loading terlebih dahulu kemudian Success atau Empty.

4. Analisis Hasil NotesScreenRobolectricTest

NotesScreenRobolectricTest berhasil menjalankan 3 test cases dalam waktu 0.016 detik yang merupakan waktu paling singkat di antara semua kelas test. Hal ini karena pengujian dilakukan pada level logika state tanpa perlu me-render UI secara penuh. Ketiga test cases berhasil memverifikasi bahwa sealed class NotesUiState berfungsi dengan benar dalam merepresentasikan berbagai kondisi UI.

F. Kendala dan Solusi

Selama pelaksanaan praktikum ini terdapat beberapa kendala yang ditemui. Pertama, build gagal dengan error 25.0.2 karena sistem menggunakan JDK versi 25 yang belum didukung oleh Gradle 8.5. Solusinya adalah mengganti JAVA_HOME ke JDK 21 yang tersedia di dalam instalasi Android Studio menggunakan perintah `$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"` di PowerShell.

Kedua, push ke GitHub ditolak karena GitHub Secret Scanning mendeteksi API key Groq yang tersimpan di file strings.xml. Solusinya adalah menghapus API key tersebut dari file, membersihkan git history menggunakan `git filter-branch`, kemudian melakukan force push ulang.

Ketiga, UI testing menggunakan device fisik terus gagal karena device sering terputus koneksinya selama proses instalasi APK test. Solusinya adalah mengganti pendekatan UI testing menggunakan Robolectric yang tidak memerlukan device fisik maupun emulator.

G. Kesimpulan

Praktikum ini berhasil mengimplementasikan Dependency Injection menggunakan Koin dengan 2 modules yang terpisah sesuai layer-nya, serta berhasil menulis 15 test cases yang seluruhnya lulus dengan success rate 100%. Penerapan DI terbukti sangat membantu dalam proses pengujian karena dependensi seperti database dan repository dapat dengan mudah diganti dengan mock object. Penggunaan MockK memudahkan pembuatan dan verifikasi mock object secara deklaratif, sementara Turbine memudahkan pengujian reactive stream dengan API yang intuitif. Secara keseluruhan, penerapan testing yang komprehensif meningkatkan kepercayaan terhadap kualitas kode dan meminimalkan risiko bug pada saat pengembangan maupun setelah deployment.